

SQLITE.LIB

Consultas SQLite em Clipper 5.2 (DOS, 16 bits), com a sintaxe e as ferramentas que você já conhece.

O Clipper 5.2 não consegue usar a biblioteca do SQLite diretamente (é código C moderno, para compiladores de 32/64 bits). Esta biblioteca resolve isso com uma ponte em C: cada consulta vira um **DBF temporário**, aberto com `dbUseArea()`. A partir daí, todos os comandos tradicionais (`GO TOP`, `SKIP`, `EOF()`, `INDEX ON`, `SEEK`, `SET FILTER`, `BROWSE`, `COPY TO...`) funcionam normalmente sobre o resultado.

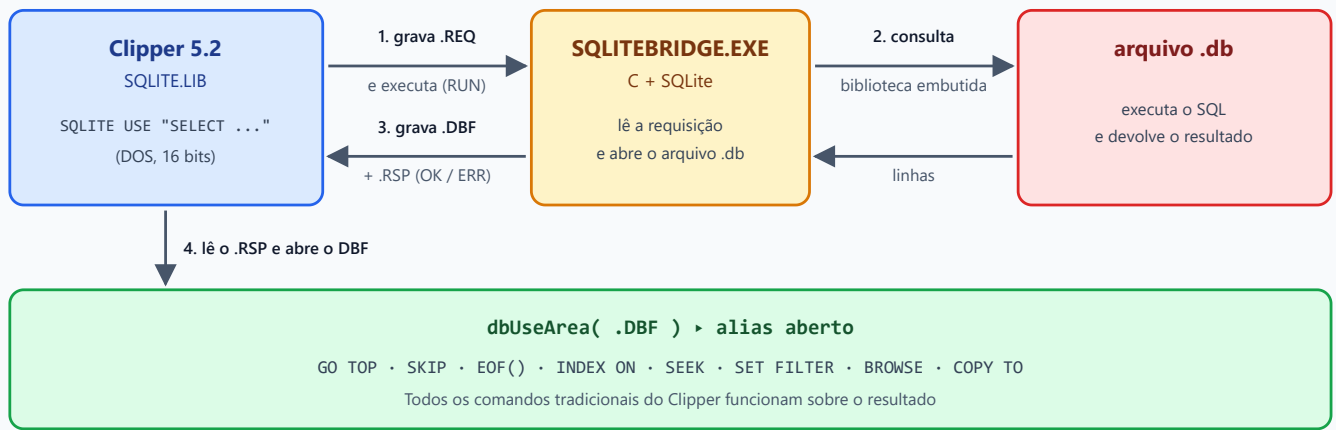
Como o SQLite não tem servidor nem usuário/senha — é só um arquivo `.db` — não há nada para instalar além do compilador: sem cliente externo, sem variáveis de ambiente.

Sumário

- [Como funciona](#)
- [Estrutura do projeto](#)
- [Início rápido](#)
- [Referência da API](#)
- [Instalação](#)
- [Conversões e limites](#)
- [Segurança](#)
- [Testes](#)

Como funciona

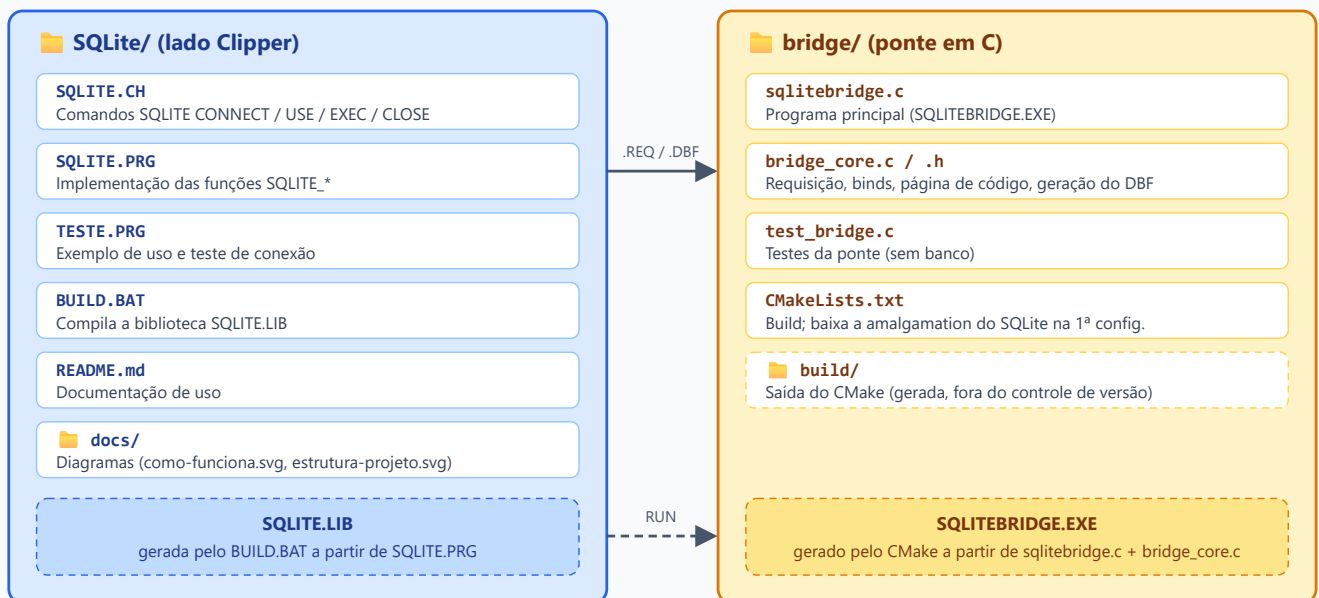
Como funciona: cada consulta vira um DBF temporário



1. A biblioteca grava um arquivo de requisição (caminho do **.db**, SQL e binds).
2. Executa **SQLiteBRIDGE.EXE** via **RUN**.
3. A ponte abre o arquivo (criando-o se ainda não existir), executa o SQL, grava o resultado em DBF e responde **OK** ou **ERR** em um arquivo **.RSP**.
4. A biblioteca abre o DBF com o alias informado.

Estrutura do projeto

Estrutura do projeto: Clipper/SQLite



Arquivo

Função

SQLite.CH

Comandos **SQLite** ... (#include "sqlite.ch")

SQLite.PRG

Implementação das funções **SQLite_***

Teste.PRG

Exemplo de uso e teste de conexão

Arquivo	Função
<code>BUILD.BAT</code>	Compila a biblioteca <code>SQLITE.LIB</code>
<code>bridge/sqlitebridge.c</code>	Programa principal da ponte (<code>SQLITEBRIDGE.EXE</code>)
<code>bridge/bridge_core.c/.h</code>	Núcleo: requisição, binds, página de código e geração do DBF
<code>bridge/test_bridge.c</code>	Testes da ponte (sem banco)
<code>bridge/CMakeLists.txt</code>	Build da ponte (baixa a amalgamation do SQLite na primeira configuração)

Início rápido

```
#include "sqlite.ch"

SQLITE CONNECT FILE "C:\DADOS\SISTEMA.DB"

SQLITE USE "SELECT * FROM F0005 WHERE DRSY = :1 AND DRRT = :2" ALIAS F0005 BIND
cSy, cRt
GO TOP
DO WHILE !Eof()
    ? F0005->DRSY, F0005->DRKY
    SKIP
ENDDO
SQLITE CLOSE F0005

SQLITE EXEC "UPDATE T SET X = :1 WHERE Y = :2" BIND 10, "A"

SQLITE DISCONNECT
```

Consultar, alterar, atualizar e apagar um registro

O DBF gerado por `SQLITE USE` é apenas uma **cópia local** do resultado: alterar o campo nele não muda o banco. Para gravar, use `SQLITE EXEC` com `UPDATE/DELETE`, filtrando pela chave do registro. Cada comando é gravado assim que termina (o SQLite roda em modo autocommit fora de uma transação explícita).

```
#include "sqlite.ch"
```

```

LOCAL cSy := "00", cRt := "UM", cKy := "ABC"
LOCAL cDesc, nAfetadas

SQLITE CONNECT FILE "C:\DADOS\SISTEMA.DB"

// 1. Consulta o registro
SQLITE USE "SELECT DRDL01 FROM F0005 WHERE DRSY = :1 AND DRRT = :2 AND DRKY = :3" ;
    ALIAS F0005 BIND cSy, cRt, cKy

IF Eof()
    ? "Registro não encontrado."
    SQLITE CLOSE F0005
    SQLITE DISCONNECT
    RETURN
ENDIF

// 2. Altera o campo (em memória)
cDesc := Upper( AllTrim( F0005->DRDL01 ) ) + " - REVISADO"
SQLITE CLOSE F0005

// 3. Atualiza a tabela no banco
nAfetadas := SQLITE_Exec( "UPDATE F0005 SET DRDL01 = :1 WHERE DRSY = :2 AND DRRT = :3 AND DRKY = :4", ;
                        { cDesc, cSy, cRt, cKy } )
IF nAfetadas == -1
    ? "Falha no UPDATE:", SQLITE_Error()
ELSE
    ? nAfetadas, "registro(s) atualizado(s)."
ENDIF

// 4. Apaga o registro
nAfetadas := SQLITE_Exec( "DELETE FROM F0005 WHERE DRSY = :1 AND DRRT = :2 AND DRKY = :3", ;
                        { cSy, cRt, cKy } )
IF nAfetadas == -1
    ? "Falha no DELETE:", SQLITE_Error()
ELSE
    ? nAfetadas, "registro(s) apagado(s)."
ENDIF

SQLITE DISCONNECT

```

Referência da API

Comandos e funções equivalentes

Comando	Função	Retorno	Descrição
SQLITE CONNECT FILE	SQLITE_Connect()	.T. / .F.	Testa o arquivo .db (criando-o se não existir) e

Comando	Função	Retorno	Descrição
<code>f</code>			guarda o caminho. Deve ser o primeiro passo antes de qualquer consulta.
<code>SQLITE USE cSql ALIAS a BIND v1, v2...</code>	<code>SQLITE_Use()</code>	<code>.T. / .F.</code>	Executa um <code>SELECT</code> e carrega o resultado em uma área de trabalho (DBF temporário) com o alias informado, para navegar com <code>DBSKIP</code> , <code>EOF()</code> etc. Os valores de <code>BIND</code> substituem <code>:1</code> , <code>:2...</code> no SQL.
<code>SQLITE EXEC cSql BIND v1, v2...</code>	<code>SQLITE_Exec()</code>	linhas afetadas, ou <code>-1</code> em erro	Executa comandos que não retornam dados (<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , <code>CREATE TABLE...</code>) e informa quantas linhas foram afetadas.
<code>SQLITE CLOSE alias</code>	—	fecha a área e apaga o DBF temporário	Encerra a consulta aberta com <code>SQLITE USE</code> e limpa o arquivo temporário criado para ela.
<code>SQLITE DISCONNECT</code>	—	descarta os dados da conexão	Encerra a sessão, esquecendo o caminho do arquivo. Use ao final do programa.

Funções auxiliares:

Função	Descrição
<code>SQLITE_Error()</code>	Texto do último erro
<code>SQLITE_Truncated()</code>	Indica se o resultado foi truncado (por exemplo, por <code>MAXROWS</code>)
<code>SQLITE_Config(cChave, xValor)</code>	Lê ou altera uma configuração; retorna o valor anterior

`SQLITE_Error()` — use logo após uma chamada que falhou:

```

IF !SQLITE_Use( "SELECT * FROM F0005", "F0005" )
    ? "Falha na consulta:", SQLITE_Error()
    RETURN
ENDIF

IF SQLITE_Exec( "UPDATE T SET X = :1 WHERE Y = :2", { 10, "A" } ) == -1
    ? "Falha no UPDATE:", SQLITE_Error()
ENDIF

```

SQLITE_Truncated() — use depois de **SQLITE_USE** para saber se o **MAXROWS** cortou o resultado:

```

SQLITE_Config( "MAXROWS", 1000 )
SQLITE_USE "SELECT * FROM F0911" ALIAS F0911

IF SQLITE_Truncated()
    ? "Atenção: só as primeiras 1000 linhas foram carregadas."
ENDIF

```

SQLITE_Config() — sem valor, lê; com valor, altera e devolve o valor anterior:

```

? SQLITE_Config( "MAXROWS" )           // 65000 (lê)

SQLITE_Config( "WORKDIR", "C:\TMP" )    // pasta dos temporários
SQLITE_Config( "BRIDGE", "C:\SQLITE\SQLITEBRIDGE.EXE" )
SQLITE_Config( "CODEPAGE", "cp437" )

// altera temporariamente e restaura depois
nAnt := SQLITE_Config( "MAXROWS", 0 )   // 0 = sem limite
SQLITE_USE "SELECT * FROM F0005" ALIAS F0005
SQLITE_Config( "MAXROWS", nAnt )

```

Configurações (**SQLITE_Config**)

Chave	Padrão	Descrição
BRIDGE	SQLITEBRIDGE.EXE	Caminho do executável da ponte
WORKDIR	%TEMP%	Pasta dos arquivos temporários (REQ, RSP, DBF)
CODEPAGE	cp850	Página de código dos textos retornados
MAXROWS	65000	Máximo de linhas por consulta (0 = sem limite)

Instalação

1. **Compilar a ponte** (CMake + compilador C; a amalgamation do SQLite é baixada do site oficial na primeira configuração):

```
cd bridge
cmake -S . -B build
cmake --build build
```

O resultado é `build\SQLITEBRIDGE.EXE` (no MinGW, executável estático).

- MinGW: use `-G "MinGW Makefiles"`.
- Sem internet: `-DFETCHCONTENT_SOURCE_DIR_SQLITE3=<pasta com sqlite3.c e sqlite3.h>`.

2. **Compilar a biblioteca** com `BUILD.BAT` (ajuste para o seu Clipper/linker) e ligar com `SQLITE.LIB`. Veja `TESTE.PRG`.
3. **Manter o `SQLITEBRIDGE.EXE` acessível**: pasta atual, `PATH` ou `SQLITE_Config("BRIDGE", ...)`.

Ambiente de execução: o Clipper precisa rodar onde o `RUN` consiga executar programas Windows. No Windows 32 bits use NTVDM, com a ponte compilada em 32 bits. No Windows 64 bits use DOSBox ou similar, com uma ponte de execução.

Diferente da versão Oracle desta biblioteca, não há Instant Client nem variáveis de ambiente para configurar: o SQLite está embutido no próprio `SQLITEBRIDGE.EXE`, e o "banco" é só o caminho de um arquivo.

Problemas comuns

Erro	Causa provável
<code>unable to open database file</code>	Pasta do arquivo <code>.db</code> não existe, ou sem permissão de escrita nela.
<code>database is locked</code>	Outro processo (outra estação, outro programa) mantém o arquivo aberto com uma transação pendente.
A ponte nao respondeu	<code>SQLITEBRIDGE.EXE</code> não está no <code>PATH</code> nem na pasta atual — ajuste com <code>SQLITE_Config("BRIDGE", ...)</code> .

Conversões e limites

Tipos SQLite → campos DBF

O SQLite tem tipagem dinâmica (cada valor guarda seu próprio tipo, independente da coluna). A ponte decide o tipo do campo pelo tipo declarado da coluna (**INTEGER**, **REAL**, **TEXT**, **DATE**..., da cláusula **CREATE TABLE**); numa expressão sem tipo declarado (**COUNT(*)**, literais...), usa o tipo do primeiro valor não nulo encontrado.

SQLite	DBF	Observação
INTEGER , REAL , NUMERIC , DECIMAL	N	INTEGER guarda até 64 bits; REAL é double (± 15 -17 dígitos significativos) — sem a precisão decimal arbitrária do NUMBER do Oracle
DATE , DATETIME , TIMESTAMP	D	Aceita texto ISO-8601 (AAAA-MM-DD...) ou INTEGER com segundos desde 1970 (epoch Unix); a hora é descartada. Coluna gravada como número juliano (REAL) não é convertida
BOOLEAN	L	Qualquer valor diferente de zero é .T.
TEXT , CHAR , VARCHAR , CLOB	C	Máx. 254; o excedente é cortado
BLOB	C	Em hexadecimal, truncado

Regras gerais

- Nomes de coluna viram nomes de campo válidos: máx. 10 caracteres, maiúsculos e únicos.
- Binds são posicionais (**:1**, **:2...**), com valores **C**, **N**, **D**, **L** ou **NIL**. Um bind **D** vira texto **"AAAA-MM-DD"** (a data vazia **CTOD("")** vira **NULL**); um bind **N** vira **INTEGER** quando não tem parte fracionária, senão **REAL**.
- Cada chamada abre e fecha o arquivo: **não há transação entre chamadas**. **SQLITE EXEC** grava (autocommit) ao final de cada comando.
- A linha de comando do **RUN** tem limite de ~127 caracteres: use caminhos curtos.

Segurança

O arquivo de requisição contém o caminho do banco (sem senha, já que o SQLite não tem) e é apagado logo após a chamada. Como o SQLite não faz autenticação, quem tiver acesso de leitura/escrita ao arquivo `.db` tem acesso completo aos dados — a segurança depende inteiramente das permissões do sistema de arquivos na pasta onde ele fica. Use uma pasta `WORKDIR` de acesso restrito para os temporários.

Testes

Os testes da ponte não precisam de banco:

```
ctest --test-dir bridge/build --output-on-failure
```

Cobrem a leitura da requisição, os binds, a conversão de página de código e a geração do DBF, e conferem que, com um arquivo `.db` inacessível, o `SQLITEBRIDGE.EXE` responde `ERR` no `.RSP`.